# CS 6220: Homework 3

**Goals**: Gain a deeper understanding of k-Nearest Neighbor (k-NN), neural networks, and the bias-variance tradeoff.

**Resources**:

- Demo notebook for neural network exploration in **Canvas**

Please submit your solution as a *single PDF file* on Gradescope by the due date and time shown there. During the submission process, tell Gradescope on which page the solution to each question is located. In general, treat this like a professional report. Not following these instructions may result in some question or the entire submission not being graded. We want our TAs to spend their time helping you learn, not fixing messy reports or searching for solutions.

This HW is worth 100 points and accounts for 0% of your final grade.

If you also want your code to be graded, please follow these instructions: As with all software projects, you must include in the root directory of your project a **README** file that briefly describes all the steps necessary to build and execute your project. This description should include all dependencies. For Python projects, create a file named **requirements.txt** in the project's root directory. Each line in this plain-text file specifies a package name and, if needed, version requirements. For other programming languages, use their equivalent of this approach.

Please start working on this homework right away, because you never know for how long you may get stuck due to an unexpected error, e.g., incompatible versions of the different software libraries used.

## Part 1: K-Nearest Neighbor

We will explore the impact of the main hyper-parameters (distance measure and k) on k-NN, but first we need a suitable dataset. Use sklearn's `make_moons`, which generates 2 interleaving half circles for a nice and easy binary classification problem. The following lines should get you started:

```
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
from sklearn.datasets import make_moons

# Generate the dataset (everyone uses the same seed for consistency)
np.random.seed(42)
X, y = make_moons(n_samples=200, noise=0.15)
```

Implement your own **k-NN classification algorithm** knn_predict(X_train, y_train, X_test, y_test, k, distance_func) as follows:

- X_train and y_train represent an entire training dataset. Ensure you can directly pass the output of make_moons to the corresponding parameters of knn_predict.
    - X_train represents the input-attribute values of the training dataset. Match its type to the corresponding output of make_moons.
    - y_train is the corresponding vector of output-attribute values. Match its type to the corresponding output of make_moons.
- Similarly, X_test and y_test represent a *single* test record. Match its type to a single record produced by make_moons.
- Parameter k is the k of k-NN.
- Parameter distance_func is k-NN's distance measure, which returns a floating-point distance value for a pair of input vectors v1, v2.
    - Instead of creating a separate distance function for Euclidean and Manhattan distance, you may implement the general $L_p$ norm, making p a parameter (p=1 for Manhattan, p=2 for Euclidean).
- The output is the majority class among the k-NN.
    - If there are ties, resolve them in a "meaningful" way.

Choose your own strategy for finding the k training records closest to the test record. It is fine (but possibly slow) to check each training record one-by-one. Optionally, you can explore index structures to speed up this process. However, please be aware that indexing in multiple dimensions may not be much faster, unless one allows for approximate answers. We want to find the *exact* k-nearest neighbors.

Implement a function that **visualizes a 2-dimensional k-NN model** as follows:

- It should support 2 numerical input attributes and binary output.
- Given a set of training records, it should plot each record at the corresponding position and use different colors and/or markers to represent the classes.
- It should visualize the decision boundary using the appropriate visual elements.
    - You may run a regular grid of 100x100 inputs through the k-NN model, marking the corresponding point in the plot with the color representing the predicted class.
    - Feel free to pick alternative visualizations that may be more elegant.

Implement a function that **computes training and test error** for a k-NN model as follows:

- Predict for all training records. The training error is the ratio of the number of mispredicted training records to the number of all training records.
- For test error, perform leave-one-out cross validation (LOOCV) on the dataset as follows:
    - For each record R do the following:
        - Use R as the test record and all others as training records.
        - Compare predicted and true class for R.
    - Report the test error as the ratio of the number of mispredicted test records to the number of all test records.

## Part 2: Neural Network

Write a function `train_perceptron(training_record, learning_rate, w1, w2, b)` that implements the perceptron training rule as discussed in the lecture, updating weights and bias for a single training record:

- A training record has schema (x1, x2, y), i.e., 2 input-attribute values and 1 output-attribute value that is either 0 or 1. Hence the perceptron has 2 inputs and 1 output.
    - This means that there are 2 weight parameters w1, w2 and a bias parameter b.
- Parameters w1, w2, and b define the function w1*x1 + w2*x2 + b of the perceptron.

Write a function `train_neuron(training_record, learning_rate, w, b, cost_derivative, activation_derivative)` to train a simple single-unit neural "network" by updating weights and bias based on a single training record:

- A training record has schema (x,y), i.e., 1 input-attribute value and 1 output-attribute value. Hence the neuron has 1 input x and 1 output y.
    - This means that there are 1 weight parameter w and a bias parameter b.
- Parameters w and b define the function w*x+b of the neuron.
- Parameter cost_derivative is the derivative *function* of the cost function. It should support squared error and cross-entropy.
- Parameter activation_derivative is the derivative *function* of the activation function. It should support sigmoid, tanh, and ReLU. (For ReLU, set the derivative at zero to zero.)

## Report

Write a brief report about your findings, answering the following questions:

1. [15 points] Show the link to your repository with source code (k-NN, neural networks) on GitHub. Make sure the link works and your code is well documented. Use markup cells in the Jupyter notebook to explain the main structure of your solution, including helpful headings. In addition, use comments in code cells to explain important program aspects. **Before committing and pushing the final version of your notebook, run all relevant cells so that their output shows up in the committed file.**

2. [15 points] Show the pseudo-code for k-NN. If you used AI to generate code, include a **link to the corresponding chat history**. (GenAI services generally allow you to share a link to each individual chat. This should make it easier for you to show us how you interacted with the AI, compared to manually selecting all the major prompts.) As usual, your prompts should reflect the main steps in the pseudo-code. If you did not use AI, we expect to see a "natural" progression of diffs in your code repo, matching "manual" code development.
3. [8 points] Create a dataset using `make_moons` with the following parameters: np.random.seed(42) and X, y = make_moons(n_samples=200, noise=0.15). Then explore the predictions for test points (0.40, 0.45), (0.08, 0.78), and (0.75, -0.15). For each test point, for both distance metrics (Euclidean and Manhattan), and for k= 1, 3, 5:
    a. Report the number of votes for each class.
    b. State the predicted class.
    c. Do the choice of k and the distance measure affect the predictions?
    d. Create a **minimal** example to demonstrate how Euclidean and Manhattan distance can cause different predictions even when training data and test record are the same.
4. [4 points] Use your visualization function to plot the 1-NN model for the make_moon data. Then plot the 10-NN model separately. Comparing the plots, can you tell which model has higher variance? Briefly justify your answer.
5. [8 points] Answer each of the following questions precisely and concisely:
    a. Why would we generally expect training error to decrease when decreasing k?
    b. Is it possible for training error to reach zero? How should we set k for this?
    c. Can you construct a minimal example to demonstrate why for some data, it is impossible to reach a training error of zero? Show it.
    d. Assume we increase k from 1 to the cardinality of the training data. What would we expect for the trend of training and test error? Briefly justify your answer. Use your program to try it out for Euclidean distance.
6. [15 points] Show the pseudo-code for `train_perceptron` and `train_neuron`. If you used AI to generate code, include a **link to the corresponding chat history**. (GenAI services generally allow you to share a link to each individual chat. This should make it easier for you to show us how you interacted with the AI, compared to manually selecting all the major prompts.) As usual, your prompts should reflect the main steps in the pseudo-code. If you did not use AI, we expect to see a "natural" progression of diffs in your code repo, matching "manual" code development.
7. [4 points] Manually create dataset [(0.9, 1.1, 0), (1.0, 0.8, 0), (1.1, 1.0, 0), (0.8, 1.0, 0), (1.2, 1.1, 0), (3.1, 2.9, 1), (3.0, 3.2, 1), (2.9, 3.0, 1), (3.2, 3.1, 1), (3.0, 2.8, 1)] of schema (x1, x2, y). Produce an x1-x2 scatterplot of the dataset, indicating the class by color/marker. Then, starting with a bad initial decision boundary (pick w1, w2, b so that there are as many prediction errors as possible on your data), repeatedly call `train_perceptron` on the training records one-by-one, each time plotting the new decision boundary. Report the error on the entire training dataset after each call. Select a learning rate for which "some learning clearly happens", maybe try 0.1 first. Show a few plots of consecutive calls that indicate how the perceptron learns. Note that the first `train_perceptron` call takes the initial (w1, w2, b) combination, but later calls get passed the (w1,

w2, b) from the previous call. Plot the model found that has minimal training error. Does this model perfectly separate the 2 classes?

8. [4 points] Now add record (1.0, 1.0, 1) to your dataset. Re-run the perceptron training experiment and report what happens. Briefly explain the reason for this behavior.
9. [15 points] Use your data generator from HW 2 to produce 100 records of schema (x,y) for true function y=2x+3, x in [0,1], and noise level 0.1. Run `train_neuron` on this dataset to train a regression model, exploring different activation functions and cost functions.
    a. For different combinations of activation and cost function, create a movie-like visualization that shows in an x-y plot the true function and how the model changes in each iteration. You can use matplotlib.animation.FuncAnimation and HTML(ani.to_jshtml()) to do this. AI and your classmates on the discussion board can help you with syntax and selecting the right tools. Make sure at least one of these animations appears in your submitted Jupyter notebook.
    b. Which activation and cost function seem to work best? How can you tell?
    c. Try different learning rates, e.g., 0.001, 0.01, 0.1, 1, 10. How does this affect model quality and convergence? Show a couple of extreme examples to make your case.
    d. Augment your code to return the gradient for both w and b after each iteration. Show a plot of these numbers. How does the gradient change from epoch to epoch? Why does this happen?
    e. What would happen if you used a linear activation function?
10. [4 points] Run the multi-layer perceptron from the attached demo notebook (see resources). How many layers are in the model, and what activation function is used for each layer?
11. [4 points] Explain the generated plot. What is the plot showing? What does the shape of the curve tell you? What does the difference between the curves tell you? Is the network learning?
12. [4 points] Explore different activation and cost functions, in particular cross-entropy cost. How do the gradients change now? When does the network learn best?

## Important Notes

This should not be necessary for this assignment, but if you want to share large (data) files with instructor and TAs, do **not** add them to your GitHub project. Instead, do the following:

- Create a new folder in your **Northeastern OneDrive** and share it with TAs and instructor, but no one else.
- In that folder, create a subfolder for each homework.
- Copy into that subfolder the big files you want to share.
- Double-check the sharing settings. (By default, files inherit the top-level folder's permissions.)

The **submission time** of your homework is the *latest timestamp of any of the deliverables included*. For the PDF it is the time reported by Gradescope; for the files on GitHub it is the time the files were pushed to GitHub, according to GitHub. If you want to keep things simple, do the following:

1. Push/copy all requested files to GitHub and make sure everything is there. (Optional: Create a release with a version number for this snapshot of your repository.)
2. Submit the report on Gradescope. Open the submitted file to verify everything is okay.
3. Do not push any more changes to the files for this HW on GitHub.